

```
# Selection Sort - Question 1: Trophy Ranking System
def top_k_scores(arr, k):
    arr = arr[:]
    n = len(arr)

    for i in range(min(k, n)):
        max_index = i
        for j in range(i + 1, n):
            if arr[j] > arr[max_index]:
                max_index = j
        arr[i], arr[max_index] = arr[max_index], arr[i]

    return arr[:min(k, n)]

# Demonstration
print("\n--- Selection Sort - Question 1: Trophy Ranking System ---")
trophy_scores = [150, 200, 100, 250, 180]
k = 3
top_scores = top_k_scores(trophy_scores, k)
print(f"Original scores: {trophy_scores}")
print(f"Top {k} scores: {top_scores}")
```

```
--- Selection Sort - Question 1: Trophy Ranking System ---
Original scores: [150, 200, 100, 250, 180]
Top 3 scores: [250, 200, 180]
```

```
# Selection Sort - Question 2: Memory-Constrained Embedded Device

def selection_sort_min_writes(arr):
    arr = arr[:]
    swaps = 0
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            swaps += 1

    return arr, swaps

# Demonstration
print("\n--- Selection Sort - Question 2: Memory-Constrained Embedded Device ---")
device_data = [5, 2, 8, 1, 9, 4]
sorted_data, num_swaps = selection_sort_min_writes(device_data)
print(f"Original data: {device_data}")
print(f"Sorted data: {sorted_data}, Swaps: {num_swaps}")
```

```
--- Selection Sort - Question 2: Memory-Constrained Embedded Device ---
Original data: [5, 2, 8, 1, 9, 4]
Sorted data: [1, 2, 4, 5, 8, 9], Swaps: 3
```

```
# Selection Sort - Question 3: Library Book Reordering
def reorder_shelf(arr):
    arr = arr[:]
    moves = 0
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            moves += 1

    return arr, moves
```

```
# Demonstration
print("\n--- Selection Sort - Question 3: Library Book Reordering ---")
books_shelf = ["War and Peace", "1984", "To Kill a Mockingbird", "Pride and Prejudice"]
sorted_shelf, num_moves = reorder_shelf(books_shelf)
print(f"Original shelf: {books_shelf}")
print(f"Reordered shelf: {sorted_shelf}, Moves: {num_moves}")
```

```
--- Selection Sort - Question 3: Library Book Reordering ---
Original shelf: ['War and Peace', '1984', 'To Kill a Mockingbird', 'Pride and Prejudice']
Reordered shelf: ['1984', 'Pride and Prejudice', 'To Kill a Mockingbird', 'War and Peace'], Moves: 2
```

```
# Selection Sort - Question 4: Contest Prize Distribution
def distribute_prizes(participants):
    participants = participants[:]
    n = len(participants)

    for i in range(n - 1):
        max_index = i
        for j in range(i + 1, n):
            if participants[j][1] > participants[max_index][1]:
                max_index = j
        participants[i], participants[max_index] = participants[max_index], participants[i]

    return participants
```

```
# Demonstration
print("\n--- Selection Sort - Question 4: Contest Prize Distribution ---")
contest_participants = [("Alice", 100), ("Bob", 150), ("Charlie", 120), ("David", 80)]
sorted_participants = distribute_prizes(contest_participants)
print(f"Original participants: {contest_participants}")
print(f"Distributed prizes: {sorted_participants}")
```

```
--- Selection Sort - Question 4: Contest Prize Distribution ---
Original participants: [('Alice', 100), ('Bob', 150), ('Charlie', 120), ('David', 80)]
Distributed prizes: [('Bob', 150), ('Charlie', 120), ('Alice', 100), ('David', 80)]
```

```
# Selection Sort - Question 5: Small Dataset in a Mobile App
def selection_sort(arr):
    arr = arr[:]
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr
```

```
# Demonstration
print("\n--- Selection Sort - Question 5: Small Dataset in a Mobile App ---")
mobile_data = [30, 10, 50, 20, 40]
sorted_mobile_data = selection_sort(mobile_data)
print(f"Original mobile data: {mobile_data}")
print(f"Sorted mobile data: {sorted_mobile_data}")
```

```
--- Selection Sort - Question 5: Small Dataset in a Mobile App ---
Original mobile data: [30, 10, 50, 20, 40]
Sorted mobile data: [10, 20, 30, 40, 50]
```

```
# Bubble Sort - Question 1: Exam Result Slip Correction
def optimized_bubble_sort(arr):
    arr = arr[:]
    n = len(arr)
    passes = 0

    for i in range(n - 1):
        swapped = False
        passes += 1

        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
```

```

        arr[j], arr[j + 1] = arr[j + 1], arr[j]
        swapped = True

    if not swapped:
        break

    return arr, passes

# Demonstration
print("\n--- Bubble Sort - Question 1: Exam Result Slip Correction ---")
exam_scores = [75, 60, 90, 80, 70]
sorted_scores, passes = optimized_bubble_sort(exam_scores)
print(f"Original exam scores: {exam_scores}")
print(f"Sorted exam scores: {sorted_scores}, Passes: {passes}")

```

```

--- Bubble Sort - Question 1: Exam Result Slip Correction ---
Original exam scores: [75, 60, 90, 80, 70]
Sorted exam scores: [60, 70, 75, 80, 90], Passes: 4

```

```

# Bubble Sort - Question 2: Traffic Signal Priority Queue
def bubble_sort_queue(queue):
    priority = {
        "ambulance": 1,
        "bus": 2,
        "car": 3
    }

    queue = queue[:]
    n = len(queue)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            if priority[queue[j]] > priority[queue[j + 1]]:
                queue[j], queue[j + 1] = queue[j + 1], queue[j]

    return queue

# Demonstration
print("\n--- Bubble Sort - Question 2: Traffic Signal Priority Queue ---")
traffic_queue = ["car", "bus", "ambulance", "car", "bus"]
sorted_queue = bubble_sort_queue(traffic_queue)
print(f"Original traffic queue: {traffic_queue}")
print(f"Sorted traffic queue: {sorted_queue}")

```

```

--- Bubble Sort - Question 2: Traffic Signal Priority Queue ---
Original traffic queue: ['car', 'bus', 'ambulance', 'car', 'bus']
Sorted traffic queue: ['ambulance', 'bus', 'bus', 'car', 'car']

```

```

# Bubble Sort - Question 3: Bubble Sort Visualization Tool
def bubble_sort_with_frames(arr):
    arr = arr[:]
    frames = [arr[:]]
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
        frames.append(arr[:])

    return frames

# Demonstration
print("\n--- Bubble Sort - Question 3: Bubble Sort Visualization Tool ---")
visualization_arr = [3, 1, 4, 2]
frames = bubble_sort_with_frames(visualization_arr)
print(f"Original array for visualization: {visualization_arr}")
print(f"Visualization frames: {frames}")

```

```

--- Bubble Sort - Question 3: Bubble Sort Visualization Tool ---
Original array for visualization: [3, 1, 4, 2]
Visualization frames: [[3, 1, 4, 2], [1, 3, 2, 4], [1, 2, 3, 4], [1, 2, 3, 4]]

```

```
# Bubble Sort - Question 4: Sorting Sensor Alerts by Severity
def bubble_sort_plain(arr):
    arr = arr[:]
    comparisons = 0
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr, comparisons

def bubble_sort_optimized(arr):
    arr = arr[:]
    comparisons = 0
    n = len(arr)

    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr, comparisons

# Demonstration
print("\n--- Bubble Sort - Question 4: Sorting Sensor Alerts by Severity ---")
sensor_alerts = [5, 1, 4, 2, 8]
sorted_plain, comparisons_plain = bubble_sort_plain(sensor_alerts)
sorted_optimized, comparisons_optimized = bubble_sort_optimized(sensor_alerts)
print(f"Original sensor alerts: {sensor_alerts}")
print(f"Sorted (plain): {sorted_plain}, Comparisons: {comparisons_plain}")
print(f"Sorted (optimized): {sorted_optimized}, Comparisons: {comparisons_optimized}")
```

```
--- Bubble Sort - Question 4: Sorting Sensor Alerts by Severity ---
Original sensor alerts: [5, 1, 4, 2, 8]
Sorted (plain): [1, 2, 4, 5, 8], Comparisons: 10
Sorted (optimized): [1, 2, 4, 5, 8], Comparisons: 9
```

```
# Bubble Sort - Question 5: Card Game Hand Sorting
def bubble_sort_hand(hand):
    hand = hand[:]
    n = len(hand)
    passes = 0

    for i in range(n - 1):
        swapped = False
        passes += 1

        for j in range(n - 1 - i):
            if hand[j] > hand[j + 1]:
                hand[j], hand[j + 1] = hand[j + 1], hand[j]
                swapped = True

        if not swapped:
            break

    return hand, passes

# Demonstration
print("\n--- Bubble Sort - Question 5: Card Game Hand Sorting ---")
card_hand = ["King", "2", "Ace", "Queen", "Jack"]
sorted_hand, card_passes = bubble_sort_hand(card_hand)
print(f"Original card hand: {card_hand}")
print(f"Sorted card hand: {sorted_hand}, Passes: {card_passes}")
```

```
--- Bubble Sort - Question 5: Card Game Hand Sorting ---
```

```
Original card hand: ['King', '2', 'Ace', 'Queen', 'Jack']
Sorted card hand: ['2', 'Ace', 'Jack', 'King', 'Queen'], Passes: 3
```

```
# Insertion Sort - Question 1: Live Leaderboard Updates
def insert_updated_score(board, score):
    board = board[:]
    board.append(score)
    shifts = 0

    i = len(board) - 2

    while i >= 0 and board[i] < score:
        board[i + 1] = board[i]
        shifts += 1
        i -= 1

    board[i + 1] = score
    return board, shifts

# Demonstration
print("\n--- Insertion Sort - Question 1: Live Leaderboard Updates ---")
leaderboard = [80, 95, 70]
new_score = 85
updated_leaderboard, shifts = insert_updated_score(leaderboard, new_score)
print(f"Original leaderboard: {leaderboard}")
print(f"Updated leaderboard with score {new_score}: {updated_leaderboard}, Shifts: {shifts}")
```

```
--- Insertion Sort - Question 1: Live Leaderboard Updates ---
Original leaderboard: [80, 95, 70]
Updated leaderboard with score 85: [80, 95, 85, 70], Shifts: 1
```

```
# Insertion Sort - Question 2: Card Sorting While Playing
def pick_up_card(hand, card):
    hand = hand[:]
    hand.append(card)

    i = len(hand) - 2

    while i >= 0 and hand[i] > card:
        hand[i + 1] = hand[i]
        i -= 1

    hand[i + 1] = card
    return hand

# Demonstration
print("\n--- Insertion Sort - Question 2: Card Sorting While Playing ---")
player_hand = [5, 10, 2]
new_card = 7
sorted_hand_cards = pick_up_card(player_hand, new_card)
print(f"Original player hand: {player_hand}")
print(f"Hand after picking up {new_card}: {sorted_hand_cards}")
```

```
--- Insertion Sort - Question 2: Card Sorting While Playing ---
Original player hand: [5, 10, 2]
Hand after picking up 7: [5, 10, 2, 7]
```

```
#Insertion Sort - Question 3: Real-Time Stock Price Feed

def insert_price(prices, price):
    prices = prices[:]
    prices.append(price)

    i = len(prices) - 2

    while i >= 0 and prices[i] > price:
        prices[i + 1] = prices[i]
        i -= 1

    prices[i + 1] = price
    return prices

# Demonstration
print("\n--- Insertion Sort - Question 3: Real-Time Stock Price Feed ---")
```

```

stock_prices = [100.5, 99.2, 101.8]
new_price = 100.0
updated_prices = insert_price(stock_prices, new_price)
print(f"Original stock prices: {stock_prices}")
print(f"Prices after inserting {new_price}: {updated_prices}")

```

```

--- Insertion Sort - Question 3: Real-Time Stock Price Feed ---
Original stock prices: [100.5, 99.2, 101.8]
Prices after inserting 100.0: [100.5, 99.2, 100.0, 101.8]

```

```

# Insertion Sort - Question 4: Playlist Reordering by Recently Added
def insert_song(playlist, song):
    playlist = playlist[:]
    playlist.append(song)

    i = len(playlist) - 2

    while i >= 0 and playlist[i][1] > song[1]:
        playlist[i + 1] = playlist[i]
        i -= 1

    playlist[i + 1] = song
    return playlist

# Demonstration
print("\n--- Insertion Sort - Question 4: Playlist Reordering by Recently Added ---")
music_playlist = [("Song A", 5), ("Song B", 10), ("Song C", 3)]
new_song = ("Song D", 8)
updated_playlist = insert_song(music_playlist, new_song)
print(f"Original playlist: {music_playlist}")
print(f"Playlist after adding {new_song}: {updated_playlist}")

```

```

--- Insertion Sort - Question 4: Playlist Reordering by Recently Added ---
Original playlist: [('Song A', 5), ('Song B', 10), ('Song C', 3)]
Playlist after adding ('Song D', 8): [('Song A', 5), ('Song B', 10), ('Song C', 3), ('Song D', 8)]

```

```

# Insertion Sort - Question 5: Nearly Sorted Sensor Log Correction
def insertion_sort_count_shifts(arr):
    arr = arr[:]
    shifts = 0

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            shifts += 1
            j -= 1

        arr[j + 1] = key

    return arr, shifts

# Demonstration
print("\n--- Insertion Sort - Question 5: Nearly Sorted Sensor Log Correction ---")
sensor_log = [10, 12, 9, 15, 11]
sorted_log, log_shifts = insertion_sort_count_shifts(sensor_log)
print(f"Original sensor log: {sensor_log}")
print(f"Sorted sensor log: {sorted_log}, Shifts: {log_shifts}")

```

```

--- Insertion Sort - Question 5: Nearly Sorted Sensor Log Correction ---
Original sensor log: [10, 12, 9, 15, 11]
Sorted sensor log: [9, 10, 11, 12, 15], Shifts: 4

```

